Reinforcement Learning Agents for Text Flappy Bird

Chien-Wei Weng

CentraleSupélec, Université Paris-Saclay
MSc Data Sciences and Business Analytics
chien-wei.weng@student-cs.fr

Abstract. This report presents the implementation and comparison of two reinforcement learning agents — GLIE Every-Visit Monte Carlo (MC) and Semi-Gradient SARSA(λ) — applied to the Text Flappy Bird environment. MC uses a tabular Q-table updated from complete episode returns, while SARSA(λ) uses tile-coded function approximation with eligibility traces updated at every step. SARSA(λ) converges significantly faster and achieves a mean greedy score of 185.0 steps survived versus 8.9 for MC under default parameters. Parameter sweeps of SARSA(λ) reveal a critical α–λ interaction: reducing α from 0.1 to 0.01 with λ=0.9 improves greedy performance from 185.0 to 473.3 steps. Generalisation tests show SARSA(λ) transfers robustly to easier configurations but degrades gracefully on harder ones, while MC exhibits zero generalisation to unseen state ranges.

1 Experimental Setup

Environment. Both agents are trained on `TextFlappyBird-v0` with configuration `height=15, width=20, pipe_gap=4`. The observation is a tuple `(dx, dy)` representing the bird's distance to the nearest pipe gap center, with declared ranges $dx \in [0, 13]$ and $dy \in [-11, 10]$. The action space is binary: 0 (fall) or 1 (flap). The reward is +1 per timestep survived; episodes terminate on collision with no additional terminal penalty observed in practice.

State representation. MC uses a tabular Q-table mapping `(dx, dy)` tuples to Q-value arrays, totalling 308 state-action entries. SARSA(λ) uses tile coding over `(dx, dy)` with 8 tilings of 8×8 tiles each, producing a 512-dimensional sparse binary feature vector. The Q-value is approximated linearly as $\hat{q}(s, a) = \mathbf{w}_a \cdot \mathbf{x}(s)$.

Agent 1 — GLIE Every-Visit MC. At episode end, returns are computed backwards and Q-values updated as $Q(s, a) \leftarrow Q(s, a) + \frac{1}{N(s,a)}\big(G_t - Q(s, a)\big)$. The exploration schedule follows GLIE decay: $\varepsilon = 1/k$ for episode k, ensuring convergence guarantees while eliminating exploration asymptotically.

Agent 2 — Semi-Gradient SARSA(λ). Weights are updated at every step using the TD error $\delta = R + \gamma\hat{q}(S', A'; \mathbf{w}) - \hat{q}(S, A; \mathbf{w})$ and eligibility trace

[0] Code available at `https://github.com/wengchienwei/rl-text-flappy-bird`

$\mathbf{z} \leftarrow \gamma\lambda\mathbf{z} + \mathbf{x}(S, A)$, yielding $\mathbf{w} \leftarrow \mathbf{w} + \alpha\delta\mathbf{z}$. A fixed $\varepsilon = 0.1$ is used throughout. Weight clipping at $\pm 10^6$ prevents numerical overflow at large α values.

Training. Both agents train for 5000 episodes with $\gamma = 0.99$. Default hyperparameters: MC uses GLIE ε decay; SARSA(λ) uses $\alpha = 0.1$, $\lambda = 0.9$. A best configuration ($\alpha = 0.01$, $\lambda = 0.9$) is identified via parameter sweep and retrained for 5000 episodes. Table 1 summarises the key differences between the two agents.

Table 1: Comparison of MC and SARSA(λ) agents.

Property	MC	SARSA(λ) default	SARSA(λ) best
Representation	Q-table (308)	Tile coding (512-dim)	Tile coding (512-dim)
Update frequency	Per episode	Per step	Per step
ε schedule	$1/k$ (GLIE)	0.1 (fixed)	0.1 (fixed)
α	—	0.1	0.01
Convergence	∼1500 ep	∼500 ep	∼2000 ep
Greedy eval mean	8.9	185.0	473.3

SARSA(λ) converges faster due to online per-step updates combined with eligibility traces that propagate credit backwards through recent states simultaneously. MC must wait until episode termination and updates each state independently without credit propagation.

2 Environment Variants and Compatibility

TextFlappyBird-v0 returns a compact `(dx, dy)` observation — two integers sufficient to determine the optimal action. The state space has 308 entries (14×22), making tabular methods tractable. A key limitation is that the environment designer has already solved the perception problem: the hand-engineered state assumes domain knowledge about which features matter. Richer agent designs tracking, for example, vertical velocity cannot be expressed within this observation.

TextFlappyBird-screen-v0 returns a (20×15) integer grid with values in $\{0, 1, 2\}$, yielding up to $3^{300} \approx 10^{143}$ possible states. Tabular methods and tile coding are both infeasible: the former cannot store the table, the latter cannot capture spatial structure across 300 dimensions. Non-linear function approximation (e.g. a CNN) is required.

Compatibility with original Flappy Bird. The original `flappy-bird-gym` [3] uses continuous RGB pixel observations, making both MC and tile-coded SARSA(λ) intractable for the same reasons. A CNN encoder combined with a deep RL algorithm such as DQN would be needed. The core principles — TD error, ε-greedy exploration, credit assignment — carry over, but the linear approximation $\mathbf{w} \cdot \mathbf{x}(s)$ must be replaced by a neural network.

3 Generalisation Test

Both agents trained on the default environment configuration are evaluated without retraining on three unseen configurations varying one parameter at a time. Results are shown in Table 2 and Fig. 8.

Table 2: Greedy evaluation (200 episodes, max 500 steps) across environment configurations.

Config	MC mean	SARSA(λ) mean	Interpretation
Train (gap=4)	9.0	478.9	Baseline
A: gap=6	8.8	500.0	Larger margin — transfers fully
B: gap=2	8.6	22.6	Tighter precision — degrades significantly
C: h=20, w=25	4.0	228.1	Expanded range — partial transfer

SARSA(λ) degrades gracefully: the tile coder clips out-of-range observations to the nearest boundary tile, preserving approximate behaviour. Config A improves over training since the learned policy is more precise than gap=6 demands. MC shows zero generalisation: unseen keys return $Q = 0$, so $\arg\max$ always selects action 0 (fall) — Config C produces mean=4.0, std=0.0.

4 Results

Learning curves. Per-episode scores are smoothed with a 200-episode rolling average. SARSA(λ) reaches ∼30 steps by episode 500 and plateaus due to fixed $\varepsilon = 0.1$. MC improves slowly from 6 to ∼9 steps over 5000 episodes. Fig. 1 shows the head-to-head curves; Fig. 7 shows the best configuration comparison.

State-value functions. MC's Q-value surface is sparse and patchy — unvisited states remain at $Q = 0$. SARSA(λ)'s surface is smooth due to tile coding generalisation (Fig. 2, Fig. 3). Both agents learn to flap above the gap and fall below it, but SARSA(λ)'s policy boundary is sharper and covers more of the state space.

Parameter sweeps. The λ sweep ($\alpha = 0.1$ fixed, Fig. 4) shows performance decreasing from $\lambda = 0$ (mean=70.5) to $\lambda = 0.99$ (mean=12.4). The α sweep ($\lambda = 0.9$ fixed, Fig. 5) reveals a critical α–λ interaction: traces amplify weight updates, requiring smaller α to remain stable. $\alpha = 0.01$ achieves mean=500 (episode cap) while $\alpha = 0.1$ yields mean=24.6. The near-zero std at $\lambda = 0.99$ (std=1.3) indicates policy collapse, not stability. For MC (Fig. 6), $\gamma = 0.99$ yields the best greedy score (mean=14.9); lower γ causes the agent to undervalue future survival rewards.

Best configuration. SARSA(λ) with $\alpha = 0.01$, $\lambda = 0.9$ achieves greedy mean=473.3, max=500, std=84.2 — substantially exceeding the default $\alpha = 0.1$ configuration (mean=185.0), with convergence visible after episode 1500.

References

1. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. MIT Press, Cambridge (2018)
2. Christodoulidis, S.: Text Flappy Bird Gym. GitLab repository. https://gitlab-research.centralesupelec.fr/stergios.christodoulidis/text-flappy-bird-gym (2021). Accessed 12 Mar 2026
3. Nogueira, G.: Flappy Bird Gym. GitHub repository. https://github.com/Talendar/flappy-bird-gym (2020). Accessed 12 Mar 2026

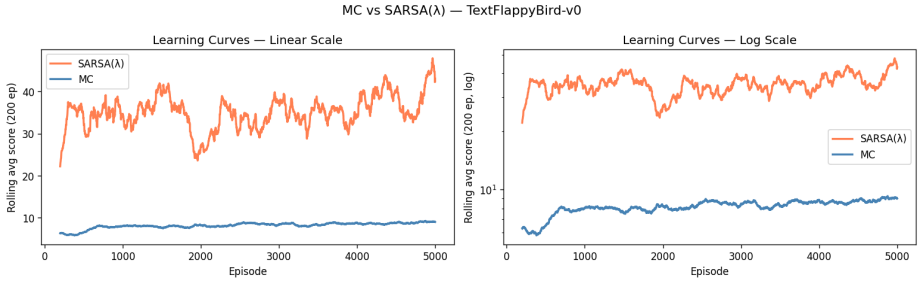

Fig. 1: Head-to-head learning curves (200-episode rolling average). Linear scale (left) shows the absolute performance gap; log scale (right) reveals MC's gradual improvement. SARSA(λ) converges within $\sim$500 episodes while MC improves slowly over 5000 episodes.

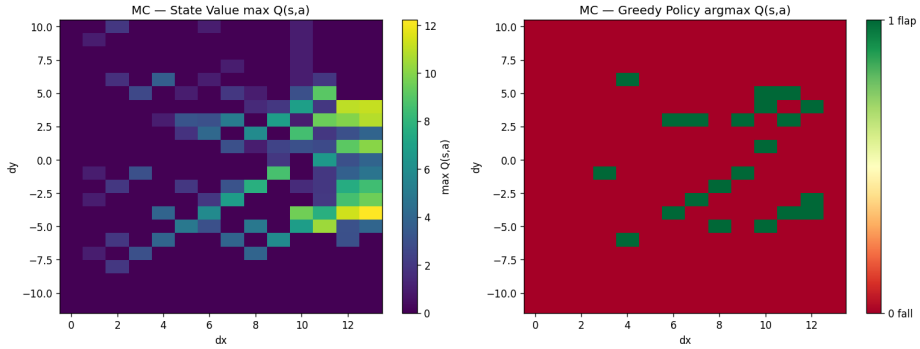

Fig. 2: MC agent: state-value estimate $\max_a Q(s,a)$ (left) and greedy policy $\arg\max_a Q(s,a)$ (right) over the (dx, dy) state space. The sparse, patchy structure reflects states never visited during training.

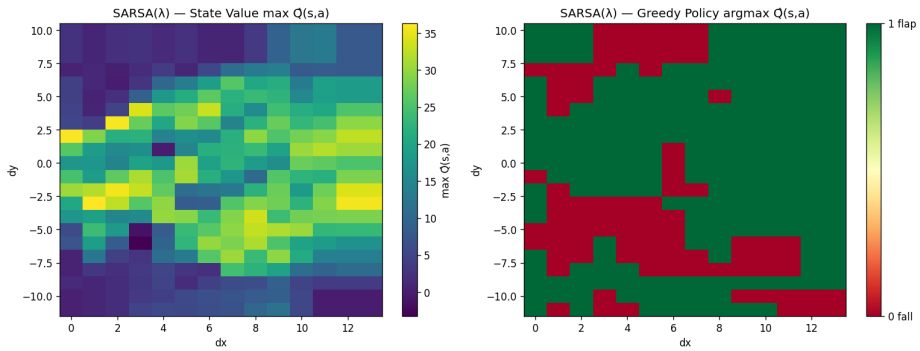

Fig. 3: SARSA(λ) agent: state-value estimate $\max_a \hat{q}(s, a; \mathbf{w})$ (left) and greedy policy (right). The smooth surface reflects tile coding generalisation — nearby states share active tiles and receive correlated weight updates.

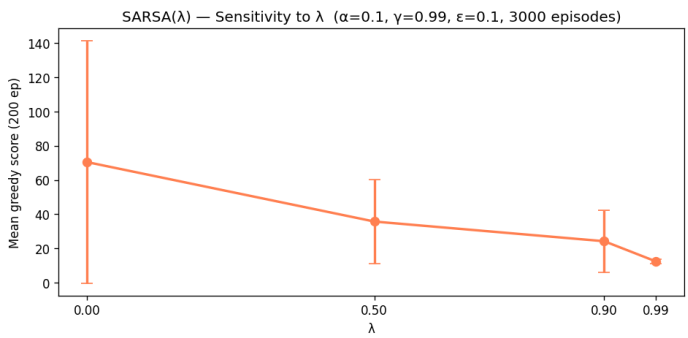

Fig. 4: SARSA(λ) sensitivity to λ ($\alpha = 0.1$, $\gamma = 0.99$, $\varepsilon = 0.1$, 3000 episodes). Performance decreases monotonically from $\lambda = 0$ to $\lambda = 0.99$. Near-zero std at $\lambda = 0.99$ indicates policy collapse, not stability.

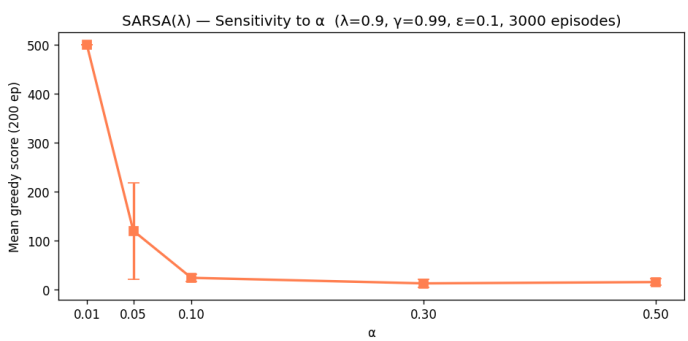

Fig. 5: SARSA(λ) sensitivity to α ($\lambda = 0.9$, $\gamma = 0.99$, $\varepsilon = 0.1$, 3000 episodes). Smaller α yields substantially better performance, demonstrating the α–λ interaction: long traces amplify updates, requiring a conservative step size.

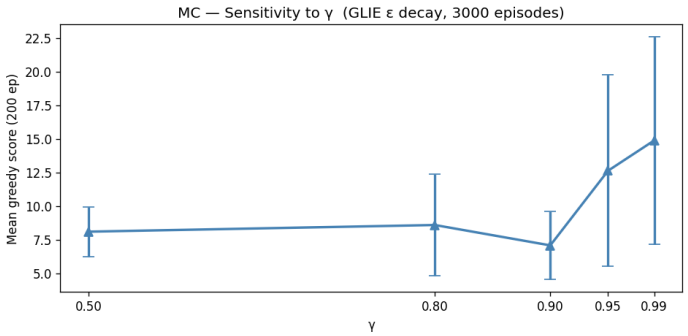

Fig. 6: MC sensitivity to γ (GLIE ε decay, 3000 episodes). Performance increases monotonically with γ, confirming that long-horizon discounting is necessary for effective survival learning.

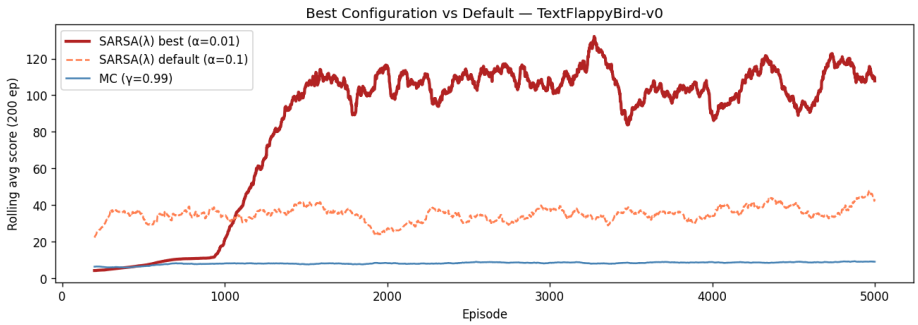

Fig. 7: Best configuration comparison: SARSA(λ) with $\alpha = 0.01$ (dark red) vs. default $\alpha = 0.1$ (dashed) vs. MC (blue). Reducing α from 0.1 to 0.01 produces a large delayed improvement, visible after episode 1500.

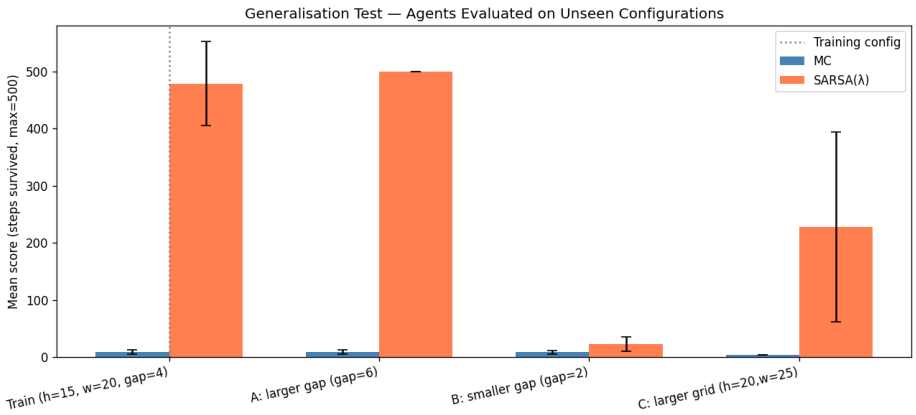

Fig. 8: Generalisation test: both agents evaluated without retraining on three unseen configurations. SARSA(λ) degrades gracefully via boundary clipping; MC collapses to mean=4.0, std=0.0 on Config C due to unseen dx values returning $Q = 0$.